

# 4

## Phase 4 - São Paulo Finalization Code (Sanitized)

This phase accepts the TGW peering from Tokyo and adds routes for cross-region connectivity.



**DEPENDENCY:** Phase 4 can only run AFTER Phase 3 completes. The peering attachment must be in `pendingAcceptance` state.

### CIDR Reference

| Region    | Resource        | CIDR                        |
|-----------|-----------------|-----------------------------|
| Tokyo     | VPC             | 10.0.0.0/16                 |
| Tokyo     | Private Subnets | 10.0.11.0/24 , 10.0.12.0/24 |
| São Paulo | VPC             | 10.1.0.0/16                 |
| São Paulo | Private Subnets | 10.1.11.0/24 , 10.1.12.0/24 |

### Step 1: Add Peering Acceptor to `04-P-main-saopaulo-tgw.tf`

ADD to existing file:

```
#####  
# ACCEPT TGW PEERING FROM TOKYO  
# Liberdade accepts corridor from Shinjuku  
#####
```

```

data "aws_ec2_transit_gateway_peering_attachment" "from_tokyo" {
  count = var.enable_tgw && var.tokyo_tgw_id != "" ? 1 : 0

  filter {
    name      = "transit-gateway-id"
    values    = [aws_ec2_transit_gateway.liberdade_tgwlab3[0].id]
  }

  filter {
    name      = "state"
    values    = ["pendingAcceptance", "available"]
  }
}

resource "aws_ec2_transit_gateway_peering_attachment_accepter" "accept_tokyo_peer01" {
  count = var.enable_tgw && var.tokyo_tgw_id != "" ? 1 : 0

  transit_gateway_attachment_id = data.aws_ec2_transit_gateway_peering_attachment.from_tokyo[0].id

  tags = {
    Name      = "liberdade-accept-tokyo-peer01"
    Type      = "Cross-Region-Peering"
    Project   = var.project_name
  }
}

#####
# TGW ROUTE TO TOKYO
# Route Tokyo CIDR (10.0.0.0/16) via peering
#####

resource "aws_ec2_transit_gateway_route" "liberdade_route_to_

```

```

tokyo" {
    count = var.enable_tgw && var.tokyo_tgw_id != "" ? 1 : 0

    destination_cidr_block      = var.tokyo_vpc_cidr # 10.
0.0.0/16
    transit_gateway_attachment_id = aws_ec2_transit_gateway_pe
ering_attachment_accepter.accept_tokyo_peer01[0].transit_gate
way_attachment_id
    transit_gateway_route_table_id = aws_ec2_transit_gateway.li
berdade_tgwlab3[0].association_default_route_table_id

    depends_on = [aws_ec2_transit_gateway_peering_attachment_ac
cepter.accept_tokyo_peer01]
}

```

## Step 2: Create 04-P-main-saopaulo-routes.tf (NEW FILE)

**What it does:** Adds VPC route so São Paulo private subnets can reach Tokyo via TGW.

```

#####
# SÃO PAULO VPC ROUTES TO TOKYO
# Liberdade private subnets → Tokyo for RDS access
# Destination: 10.0.0.0/16 (Tokyo VPC) → TGW
#####

resource "aws_route" "liberdade_to_tokyo_route01" {
    count = var.enable_tgw ? 1 : 0

    route_table_id      = aws_route_table.liberdade_private_
rtlab3.id
    destination_cidr_block = var.tokyo_vpc_cidr # 10.0.0.0/16
    transit_gateway_id     = aws_ec2_transit_gateway.liberdade_

```

```
tgwlab3[0].id
}
```

## Step 3: Update São Paulo Outputs

ADD to existing `05-outputs.tf` file:

```
output "saopaulo_peering_accepter_id" {
  description = "Sao Paulo peering accepter attachment ID"
  value       = var.enable_tgw && var.tokyo_tgw_id != "" ? aws_ec2_transit_gateway_peering_attachment_accepter.accept_tokyo_peer01[0].id : null
}

output "saopaulo_tgw_route_to_tokyo" {
  description = "TGW route to Tokyo CIDR"
  value       = var.enable_tgw && var.tokyo_tgw_id != "" ? aws_ec2_transit_gateway_route.liberdade_route_to_tokyo[0].destination_cidr_block : null
}
```

## Deployment Commands (Phase 4)

```
cd lab3/saopaulo

# Plan
terraform plan -out=phase4.tfplan

# Should show:
# - 1 data source (peering attachment lookup)
# - 1 peering accepter
# - 1 TGW route to Tokyo (10.0.0.0/16)
# - 1 VPC route to Tokyo (10.0.0.0/16)
```

```
# Apply
terraform apply phase4.tfplan
```

## Verification Checklist (Phase 4)

- ☐ Peering accepted (state: `available` in both regions)
- ☐ São Paulo TGW route table has entry for Tokyo CIDR ( `10.0.0.0/16` )
- ☐ São Paulo VPC route table ( `liberdade_private_rtlab3` ) has route to `10.0.0.0/16`

```
# Verify peering is available
aws ec2 describe-transit-gateway-peering-attachments --region sa-east-1 \
  --query "TransitGatewayPeeringAttachments[?State=='available']"
```

```
# Verify TGW route to Tokyo
aws ec2 describe-transit-gateway-route-tables --region sa-east-1
```

```
# Verify VPC route to Tokyo (10.0.0.0/16)
aws ec2 describe-route-tables --region sa-east-1 \
  --filters "Name=tag:Name,Values=*liberdade-private*" \
  --query "RouteTables[*].Routes[?DestinationCidrBlock=='10.0.0.0/16']"
```

## Post-Phase 4: Uncomment Tokyo TGW Route



**IMPORTANT:** After Phase 4 is applied and peering shows `available`, go back to Tokyo and uncomment the TGW route.

File: `04-3-tokyo.tf`

**UNCOMMENT this block:**

```
resource "aws_ec2_transit_gateway_route" "shinjuku_route_to_l
  iberdade" {
    count                                     = var.enable_tgw && var.saop
aulo_tgw_id != "" ? 1 : 0
    destination_cidr_block                 = var.saopaulo_vpc_cidr  # 1
0.1.0.0/16
    transit_gateway_attachment_id         = aws_ec2_transit_gateway_pe
ering_attachment.shinjuku_to_liberdade_peer01[0].id
    transit_gateway_route_table_id       = aws_ec2_transit_gateway.sh
injuku_tgwlab3[0].association_default_route_table_id

    depends_on = [aws_ec2_transit_gateway_peering_attachment.sh
injuku_to_liberdade_peer01]
  }
```

Then run:

```
cd lab3/tokyo
terraform plan -out=phase4b.tfplan
terraform apply phase4b.tfplan
```

## Traffic Flow Summary

```
São Paulo EC2 (10.1.11.x)
  |
  ▼
liberdade_private_rtlab3 → route 10.0.0.0/16 → TGW
  |
  ▼
liberdade_tgwlab3 (São Paulo) — TGW Peering — shinjuku_tg
wlab3 (Tokyo)
```

tlab3

(10.0.x.x)

▼  
helga\_private\_r

|

▼  
helga\_rds1ab3



**Phase 4 Complete:** Cross-region connectivity is now established. São Paulo EC2 instances in private subnets can reach `helga_rds1ab3` in Tokyo, but **NO PHI is stored outside Japan.**